

Llamadas a Función: Las instrucciones ARM

Instrucción para llamada a función:

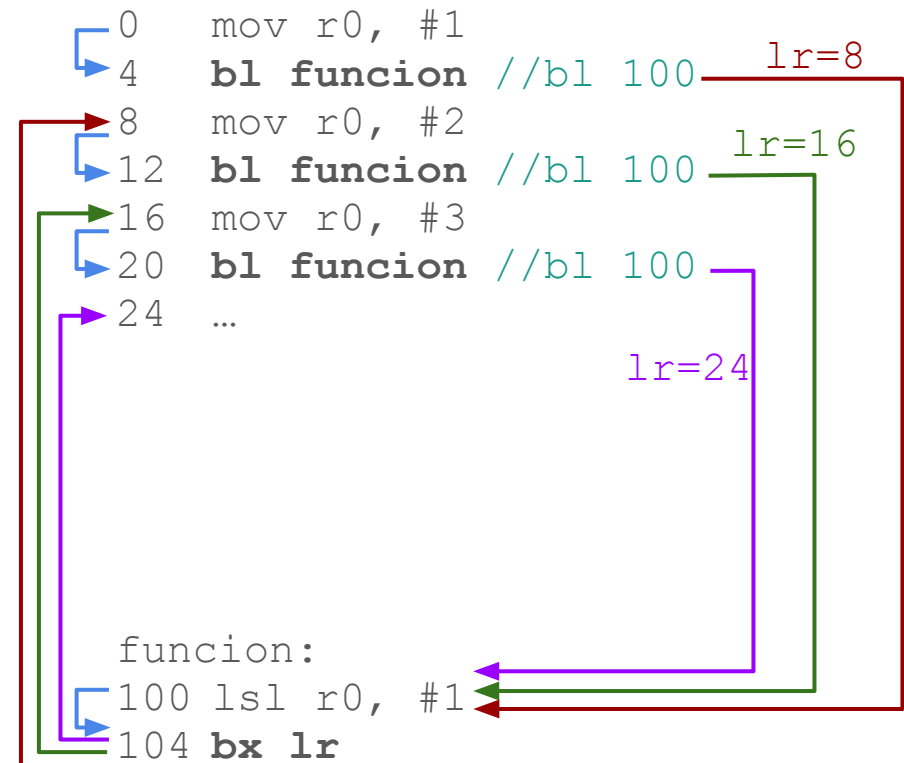
bl funcion

- Siguierte instrucción: la asociada a etiqueta **funcion**
- Guarda en registro **lr** la dirección de la instrucción posterior a **bl** (PC+4)

Instrucción para retorno de función:

bx lr

- Siguierte instrucción: la asociada a la posición de memoria almacenada en **lr**



Llamadas a Función: Convenio AAPCS

Paso de parámetros

Paso de parámetros a la función:

- Parámetro 1 → **r0**
- Parámetro 2 → **r1**
- Parámetro 3 → **r2**
- Parámetro 4 → **r3**
- Resto parámetros: por PILA

Valor de retorno de la función:

- Resultado → **r0**

```
int funcion(int a, int b, int c) {  
    return a+b+c;  
}
```

```
...  
res = funcion(5, 6, 7);  
res2 = funcion(3, res, 4);  
...
```

```
0    mov r0, #5 // 1er parámetro 5  
4    mov r1, #6 // 2º parámetro 6  
8    mov r2, #7 // 3er parámetro 7  
12   bl  funcion // resultado en r0  
16   mov r1, r0 // 2º parámetro res  
20   mov r0, #3 // 1er parámetro 3  
24   mov r2, #4 // 3er parámetro 4  
28   bl  funcion // resultado en r0  
32   ...
```

```
funcion:  
100  add r3, r0, r1 // suma 1er y 2º param.  
104  add r0, r3, r2 // suma 3er, result. en r0  
108  bx  lr
```

Llamadas a Función: Convenio AAPCS

Registros no salvados

Una función **puede modificar** libremente el contenido de los registros **NO SALVADOS**

Registros **NO SALVADOS**:

- r0
- r1
- r2
- r3
- r12

Consecuencias para el programador:

1. Cuando programes una función puedes **usar estos registros libremente** (modificando su valor como necesites)
2. Cuando llames a otra función, **da por perdidos estos registros** (no confíes que van a conservar su valor después de la llamada)

Llamadas a Función: Convenio AAPCS

Registros salvados

Una función **no puede modificar** de ninguna forma el contenido de los registros **SALVADOS**

Registros **SALVADOS**:

- r4 - r11
- SP (r13)
- LR (r14)

Consecuencias para el programador:

1. Cuando programes una función y necesites usar alguno de estos registros, tienes que:
 - a. **salvar su contenido en la PILA** antes de modificar su valor
 - b. **restaurar su valor desde la PILA** antes de salir de la función
2. Cuando llames a otra función, **el contenido de estos registros NO se modificará** (puedes confiar que van a conservar su valor después de la llamada)

Llamadas a Función: La PILA

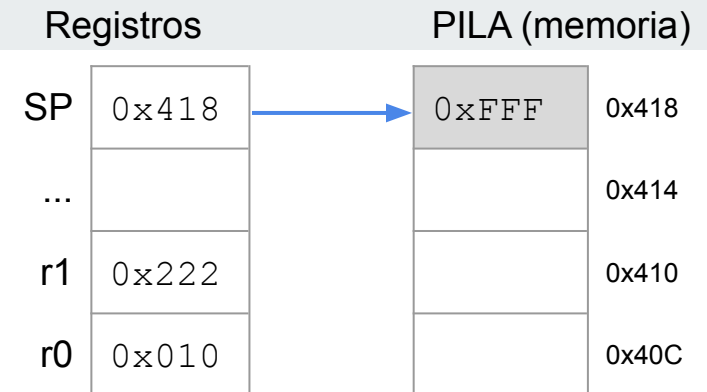
Metiendo información

PILA: zona de memoria tipo LIFO (Last In First Out)

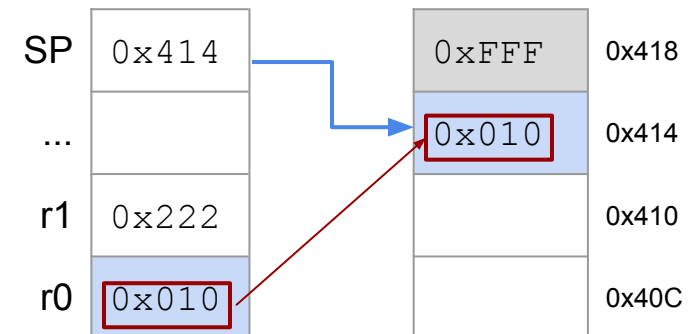
Apuntada por el **registro de pila SP**
(Stack Pointer, `r13`)

PUSH registro → Almacena valor del registro en la PILA

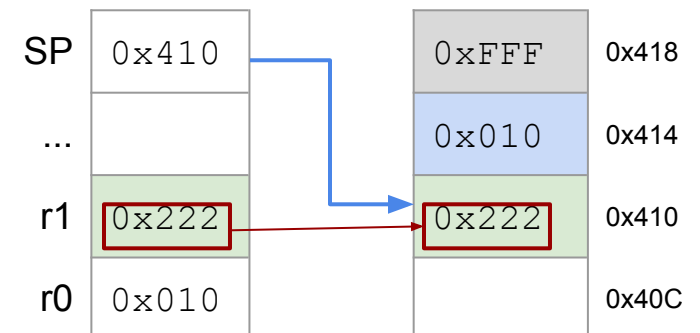
1. Decrementa SP
2. Almacena valor almacenado en registro en posición indicada por SP



PUSH { r0 }



PUSH { r1 }



Llamadas a Función: La PILA

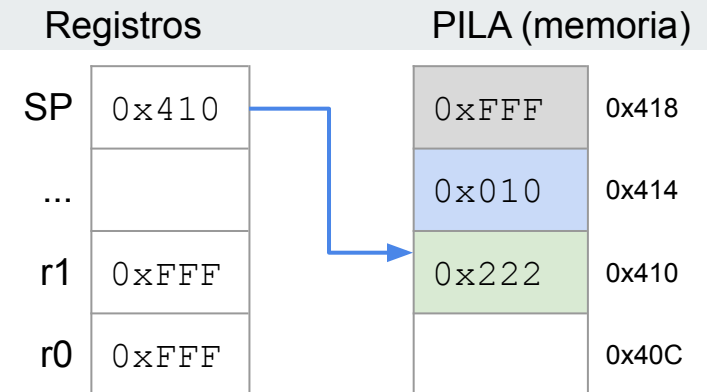
Sacando información

PILA: zona de memoria tipo LIFO (Last In First Out)

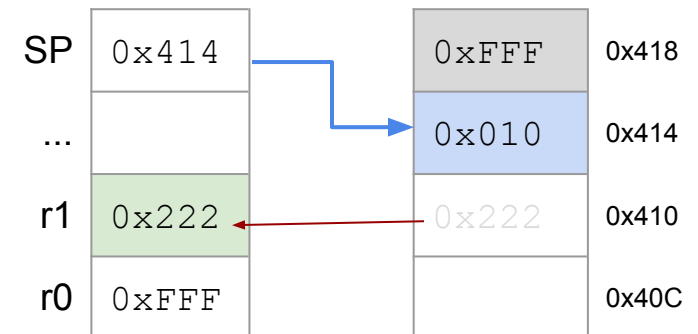
Apuntada por el **registro de pila SP**
(Stack Pointer, **r13**)

POP registro → Recupera el valor de un registro desde la PILA

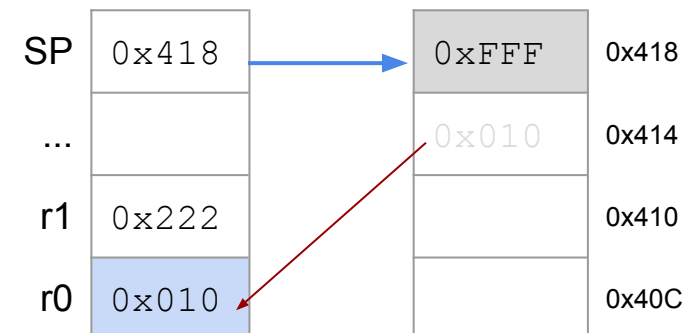
1. Lee valor de posición indicada por SP y lo almacena en registro
2. Incrementa SP



POP { r1 }



POP { r0 }



Llamadas a Función:

Ejemplo función hoja (no llama a otra)

```
void abs(int val) {  
    if (val < 0)  
        val = -val;  
    return val;  
}
```

```
abs:  
    mov r1, #0          // usamos registros  
                        // no salvados  
    cmp r0, #0          // usamos parámetro  
    sublt r0, r1, r0    // escribimos  
                        // resultado en r0  
    bx lr              // retornamos
```

1. Podemos **usar registros no salvados sin temor** ya que al no llamar a ninguna función no vamos a perder su valor
2. Si necesitamos usar **registros salvados**, los almacenamos en la PILA
3. Usamos los parámetros de entrada para lo que necesitemos
4. Escribimos en **r0** el resultado
5. Restauramos los registros salvados que hemos utilizado (si alguno)
6. Retornamos de la función